

文章笔记

文章笔记：长时间运行 Agent 的有效 Harness 设计

基于 Anthropic 工程博客整理

2026 年 4 月 2 日

Effective Harnesses for Long-Running Agents

Anthropic Engineering Blog

如何让 AI Agent 在多个 Context Window 间
持续、可靠地推进复杂任务

文章作者：Justin Young (Anthropic)

发布日期：2025-11-26

预计阅读时长：约 15 分钟

文章链接：<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

项目主页：<https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：长时间运行 Agent 的挑战	2
1.1 本章小结	2
2 问题分析：Agent 的典型失败模式	2
2.1 失败模式一：一次性尝试完成所有任务	2
2.2 失败模式二：过早宣布任务完成	3
2.3 问题的本质分解	3
2.4 本章小结	3
3 解决方案架构：双 Agent 协作模式	3
3.1 Initializer Agent：环境搭建	4
3.2 Coding Agent：增量式推进	4
3.3 信息传递机制	4
3.4 本章小结	5
4 Feature List：结构化的任务追踪	5
4.1 设计理念	5
4.2 JSON 格式的选择	5
4.3 严格的修改规则	5
4.4 本章小结	6
5 增量式开发与环境管理	6
5.1 增量式开发的关键性	6
5.2 Session 结束时的清理工作	6
5.3 Session 启动流程	7
5.4 典型的 Session 启动消息	7
5.5 本章小结	8
6 测试策略：端到端验证	8
6.1 过早标记完成的问题	8
6.2 浏览器自动化测试	8
6.3 测试工具的局限性	8
6.4 本章小结	9
7 Agent 失败模式与对策总结	9
7.1 本章小结	9
8 设计原则提炼	10
8.1 原则一：状态外部化	10
8.2 原则二：任务分解与增量推进	10
8.3 原则三：不可变的评判标准	10
8.4 原则四：先验证后推进	10
8.5 原则五：工具即能力边界	10

8.6 本章小结	10
9 未来方向与开放问题	11
9.1 单 Agent vs. 多 Agent 架构	11
9.2 领域泛化	11
9.3 本章小结	11
10 总结与延伸	11
10.1 核心贡献	11
10.2 对 Agent 开发者的启示	12
10.3 拓展阅读	12

1 引言：长时间运行 Agent 的挑战

随着 AI Agent 的能力不断增强，开发者开始要求它们承担跨越数小时甚至数天的复杂任务。然而，如何让 Agent 在多个 context window 之间保持一致的进展，仍然是一个开放的工程难题。

什么是 Context Window ?

Context window 是大语言模型在一次推理调用中能够“看到”的全部文本长度。即使像 Claude 这样的前沿模型，其 context window 也是有限的。当一个任务需要多次调用（多个 session）才能完成时，每个新 session 都从零开始——没有上一轮的记忆。这就是长时间运行 Agent 面临的核心问题。

Anthropic 工程团队的这篇博客文章提出了一个形象的比喻：想象一个软件项目由多名工程师轮班完成，但每位工程师上班时完全不记得上一班发生了什么。这正是长时间运行 Agent 面临的困境——context window 有限，而大多数复杂项目无法在单个 window 内完成。

为了解决这一问题，Anthropic 开发了一套基于 Claude Agent SDK 的双阶段方案：

1. **Initializer Agent**（初始化 Agent）：在首次运行时搭建环境；
2. **Coding Agent**（编码 Agent）：在每个后续 session 中渐进式推进任务，并为下一个 session 留下清晰的工作记录。

核心思想：像人类工程师一样交接

这套方案的关键洞察是：让 Agent 像优秀的人类软件工程师那样工作——每次交班前写好交接文档、提交代码、记录进度。通过 `claude-progress.txt` 文件配合 git history，新的 Agent session 可以快速理解当前项目状态。

1.1 本章小结

长时间运行 Agent 的核心挑战在于 context window 的有限性导致跨 session 的状态丢失。Anthropic 的方案采用“初始化 + 渐进式编码”的双阶段架构，通过结构化的环境管理和进度追踪来弥补 session 之间的断裂。

2 问题分析：Agent 的典型失败模式

在深入解决方案之前，有必要先理解 Agent 在长时间运行场景下的具体失败表现。Anthropic 团队在实验中观察到了两种主要的失败模式。

2.1 失败模式一：一次性尝试完成所有任务

即使使用了 compaction（上下文压缩）技术，前沿模型如 Opus 4.5 在面对高层次的 prompt（如“构建一个 claude.ai 的克隆”）时，仍然倾向于尝试一次性完成整个应用。这种“one-shot”行为导致：

- Agent 在实现过程中耗尽 context window
- 功能只完成了一半，且没有文档记录

- 下一个 session 的 Agent 需要花大量时间猜测之前发生了什么
- Agent 反复尝试让基础功能重新运行，浪费大量 token

Compaction 不是万能的

Compaction 是 Claude Agent SDK 的上下文管理能力，可以在 context window 接近满载时压缩历史信息。但 compaction 并不总能将完全清晰的指令传递给下一个 Agent——被压缩的信息可能丢失关键细节，导致后续 Agent 无法准确还原之前的工作状态。不能仅依赖 compaction 来解决跨 session 的连续性问题。

2.2 失败模式二：过早宣布任务完成

第二种失败模式通常出现在项目后期。当一些功能已经被构建出来后，后续的 Agent 实例会浏览项目现状，看到已有一定进展，就直接宣布整个任务完成（“declare the job done”），而实际上还有大量功能未实现。

2.3 问题的本质分解

这两种失败模式可以被分解为两个子问题：

子问题	描述
环境初始化	需要为所有待实现的功能搭建基础架构，引导 Agent 按步骤、按功能逐个推进
增量式推进	每个 Agent session 应该做出增量进展，并在结束时将环境留在一个“干净的状态”

表 1: 长时间运行 Agent 面临的两个核心子问题

什么是“干净的状态”？

“干净的状态”（clean state）指的是代码达到了可以合入主分支的标准：没有重大 bug，代码组织有序且有良好文档，新的开发者（或新的 Agent session）可以立即开始下一个功能的开发，而不需要先清理之前遗留的混乱。这与人类工程师的最佳实践完全一致。

2.4 本章小结

Agent 在长时间运行场景下有两大典型失败模式：一是贪多嚼不烂地试图一次完成所有任务，二是在项目有一定进展后过早宣布完工。这两个问题的根源都在于缺乏结构化的任务分解和进度追踪机制。

3 解决方案架构：双 Agent 协作模式

Anthropic 提出的解决方案包含两个关键角色：Initializer Agent 和 Coding Agent。两者通过共享的文件系统状态（特别是 progress 文件和 git history）实现跨 session 的信息传递。

3.1 Initializer Agent：环境搭建

Initializer Agent 仅在项目的第一个 session 中运行，负责创建后续所有 Coding Agent 所需的环境。它的具体职责包括：

- 1. 创建 `init.sh` 脚本——用于启动开发服务器等基础设施
- 2. 创建 `claude-progress.txt` 文件——记录各 Agent session 的工作日志
- 3. 生成初始 git commit——记录所有被添加的文件
- 4. 编写 feature list（功能清单）——基于用户 prompt 扩展出完整的功能需求列表

3.2 Coding Agent：增量式推进

每个后续的 session 都运行 Coding Agent，其行为模式为：

- 1. 阅读 progress 文件和 git log，了解当前进展
- 2. 从 feature list 中选择下一个待实现的功能
- 3. 实现该功能并进行测试
- 4. 提交 git commit 并更新 progress 文件
- 5. 确保环境处于干净状态后结束 session

为什么用两个不同的 Agent？

使用不同的 prompt 区分“首次运行”和“后续运行”是一个重要的设计选择。Initializer Agent 的 prompt 侧重于环境搭建和需求分析，而 Coding Agent 的 prompt 侧重于增量开发和代码质量。这种分工避免了单一 prompt 过于复杂，也让每个 Agent 能更专注于自己的职责。Anthropic 的 Claude 4 prompting guide 中也推荐了这种“第一个 context window 使用不同 prompt”的 harness 结构。

3.3 信息传递机制

两个 Agent 之间的信息传递完全依赖文件系统，而非任何内存或数据库：

信息载体	作用	设计要点
<code>claude-progress.txt</code>	记录每个 session 的工作摘要	自由文本格式，方便 Agent 快速理解上下文
<code>feature_list.json</code>	追踪所有功能的完成状态	JSON 格式，防止 Agent 随意修改内容
Git history	记录所有代码变更	描述性的 commit message，支持 revert 操作
<code>init.sh</code>	环境启动脚本	Agent 无需重新发现如何运行项目

表 2: 跨 session 信息传递的四种载体

3.4 本章小结

双 Agent 架构将”一次性完成”的不切实际目标分解为”搭建环境 + 持续增量推进”。Initializer Agent 负责创建所有必要的基础设施，Coding Agent 在每个 session 中做出可管理的增量进展。两者之间通过 progress 文件、feature list、git history 和启动脚本实现状态传递。

4 Feature List：结构化的任务追踪

Feature list 是整个方案中最关键的组件之一，它直接解决了 Agent 过早宣布完工和试图一次性完成所有任务的问题。

4.1 设计理念

Initializer Agent 会基于用户的初始 prompt，生成一份全面的功能需求清单。在”克隆 claude.ai”的示例中，这份清单包含了超过 200 个功能点，例如”用户可以打开新对话，输入问题，按回车，看到 AI 回复”。

每个功能点最初都被标记为 "passes": false，这样后续的 Coding Agent 就能清楚地看到完整功能的全貌，而不会因为部分功能已经实现就误以为项目完成。

4.2 JSON 格式的选择

```
1 {  
2   "category": "functional",  
3   "description": "New chat button creates a fresh conversation",  
4   "steps": [  
5     "Navigate to main interface",  
6     "Click the 'New Chat' button",  
7     "Verify a new conversation is created",  
8     "Check that chat area shows welcome state",  
9     "Verify conversation appears in sidebar"  
10  ],  
11   "passes": false  
12 }
```

Listing 1: Feature list 中单个功能的 JSON 结构

为什么选择 JSON 而非 Markdown？

Anthropic 团队经过实验发现，与 Markdown 文件相比，模型不太可能不恰当地修改或覆盖 JSON 文件的内容。JSON 的结构化特性使得 Agent 更倾向于只修改 passes 字段的值，而不是重写整个描述。这是一个值得借鉴的 prompt engineering 技巧：**数据格式的选择会影响模型的行为倾向。**

4.3 严格的修改规则

Coding Agent 被明确指示只能修改 passes 字段的值——从 false 改为 true。Anthropic 团队在 prompt 中使用了强措辞：

"It is unacceptable to remove or edit tests because this could lead to missing or buggy functionality."

这种措辞上的强调对于控制 Agent 行为非常有效。它明确告诉 Agent，删除或修改功能描述是不可接受的，从根本上防止了 Agent 通过降低标准来“完成”任务的取巧行为。

Agent 的“作弊”倾向

在没有明确约束的情况下，Agent 可能会通过删除或降低测试标准来更快地“完成”任务。这类似于人类中的 Goodhart 定律——当指标本身成为目标时，它就不再是一个好的指标。在 Agent 系统设计中，必须通过严格的规则和不可变的评判标准来防止这种行为。

4.4 本章小结

Feature list 通过结构化的 JSON 格式将项目需求具体化为可追踪的功能点，每个功能都有清晰的验证步骤。JSON 格式的选择、初始状态全部标记为失败、以及严格禁止修改功能描述的规则，共同构成了防止 Agent 过早宣布完工的有效机制。

5 增量式开发与环境管理

5.1 增量式开发的关键性

在具备了 feature list 的基础上，Coding Agent 被要求每次只处理一个功能（one feature at a time）。这种增量式方法被证明是解决 Agent 贪多嚼不烂问题的关键。

增量式开发带来的好处包括：

- 每个 session 的目标清晰、范围可控
- 即使某个 session 失败，损失也限制在单个功能范围内
- 通过 git commit 保留每个功能的实现记录，支持回滚
- 下一个 session 的 Agent 可以快速定位当前进展

5.2 Session 结束时的清理工作

每个 Coding Agent session 结束前必须执行以下清理操作：

1. 将代码变更提交到 git，附带描述性的 commit message
2. 在 `claude-progress.txt` 中写入本次 session 的工作摘要
3. 更新 `feature_list.json` 中已完功能的状态
4. 确保代码处于可编译、可运行的状态

Git 作为安全网

Git 不仅是版本控制工具，在这个系统中它还扮演着“安全网”的角色。当 Agent 的代码改动引入了 bug 或导致回归时，Agent 可以使用 `git revert` 回到之前的工作状态，而不是花大量时间调试。这种“先回滚再重来”的策略在长时间运行场景下尤为重要，因为每个 token 都是宝贵的资源。

5.3 Session 启动流程

每个 Coding Agent session 启动时会执行一套标准化的流程来获取当前状态：

1. 运行 `pwd` 确认工作目录
2. 阅读 `git log` 和 `progress` 文件了解近期工作
3. 阅读 `feature list`，选择下一个最高优先级的未完成功能
4. 运行 `init.sh` 启动开发服务器
5. 执行基本的端到端测试，确认现有功能正常工作
6. 确认无回归 bug 后，开始实现新功能

先测试再开发，而非先开发再测试

一个关键的设计决策是：Agent 在开始新功能开发之前，必须先验证现有功能是否正常。如果跳过这一步直接开始新功能的开发，可能会在已有 bug 的基础上引入更多问题，使情况更加恶化。这与人类工程师的最佳实践一致：pull 最新代码后先跑一遍测试，确认 CI 是绿的再开始写新代码。

5.4 典型的 Session 启动消息

以下是 Coding Agent 的典型启动行为序列：

```
1 [Assistant] I'll start by getting my bearings and understanding
2           the current state of the project.
3 [Tool Use] <bash - pwd>
4 [Tool Use] <read - claude-progress.txt>
5 [Tool Use] <read - feature_list.json>
6
7 [Assistant] Let me check the git log to see recent work.
8 [Tool Use] <bash - git log --oneline -20>
9
10 [Assistant] Now let me check if there's an init.sh script
11           to restart the servers.
12 <Starts the development server>
13
14 [Assistant] Let me verify that fundamental features are
15           still working.
16 <Tests basic functionality>
17
```

```
18 [Assistant] Core features are working correctly. Let me
19         review the feature list to determine what to
20         implement next.
21 <Starts work on a new feature>
```

Listing 2: Coding Agent 典型的 session 启动流程

这种标准化的启动流程大大减少了每个 session 的”热身”时间，避免了 Agent 浪费 token 在”弄清楚怎么运行项目”这类重复性工作上。

5.5 本章小结

增量式开发的核心是每个 session 只处理一个功能，并在 session 结束时通过 git commit 和 progress 文件将环境留在干净状态。标准化的 session 启动流程确保每个 Agent 都能快速上手，先验证再开发的策略则防止了在已有问题上叠加新问题。

6 测试策略：端到端验证

6.1 过早标记完成的问题

Anthropic 团队观察到的另一个主要失败模式是：Agent 倾向于在没有充分测试的情况下将功能标记为完成。典型的行为是——Agent 修改了代码，可能还跑了单元测试或发了几个 curl 请求，但没有验证功能是否在端到端场景下真正可用。

6.2 浏览器自动化测试

在 Web 应用开发场景下，Anthropic 团队发现让 Agent 使用浏览器自动化工具（如 Puppeteer MCP）进行端到端测试可以显著提升质量。Agent 被要求”像人类用户那样”测试功能：

- 在浏览器中打开应用
- 执行用户操作（点击、输入、导航）
- 验证 UI 响应是否符合预期
- 检查状态变更是否正确持久化

测试工具决定测试质量

提供合适的测试工具对于 Agent 的表现至关重要。当 Agent 只能通过代码审查和单元测试来验证时，它很难发现那些只在端到端场景下才暴露的 bug。而当给予 Agent 浏览器自动化能力后，它能够像真实用户一样操作应用，发现并修复仅通过代码分析无法识别的问题。**工具的能力边界决定了 Agent 的能力边界。**

6.3 测试工具的局限性

尽管浏览器自动化测试大幅提升了 Agent 的验证能力，但仍然存在一些限制：

- Claude 的视觉能力有其局限——某些 UI 细节可能无法被准确识别

- 浏览器原生组件（如 `alert` 弹窗）在 Puppeteer MCP 中不可见
- 依赖这类不可见组件的功能往往表现出更多 bug
- 复杂的动画、过渡效果难以通过自动化测试验证

工具链的可观测性

这个发现揭示了 Agent 系统设计中一个重要原则：Agent 的调试和测试能力受限于其工具链的可观测性（observability）。如果某些系统行为对 Agent 不可见（如原生弹窗、后台定时任务、网络延迟等），Agent 就无法有效地测试和修复相关问题。在设计 Agent harness 时，应该尽量扩展 Agent 的可观测范围，或者在 prompt 中明确告知这些盲区。

6.4 本章小结

端到端测试是确保 Agent 不过早标记功能完成的关键。通过浏览器自动化工具，Agent 可以像真实用户一样测试 Web 应用。但工具链的局限性意味着某些类型的 bug 仍然难以被自动发现，这是未来需要继续改进的方向。

7 Agent 失败模式与对策总结

Anthropic 团队将所有观察到的失败模式及其对策整理为以下系统性对照表：

问题	Initializer Agent 行为	Coding Agent 行为
过早宣布项目完成	基于输入 spec 创建结构化的 JSON feature list	每个 session 开始时阅读 feature list，选择单个功能开始工作
环境留有 bug 或未记录的进展	初始化 git 仓库和 progress notes 文件	session 开始时阅读 progress 文件和 git log，运行基本测试捕捉 bug；session 结束时提交 git commit 并更新 progress
过早标记功能为完成	创建包含验证步骤的 feature list	所有功能都经过端到端测试后才标记为 passing
花时间弄清楚如何运行项目	编写 <code>init.sh</code> 启动脚本	session 开始时阅读 <code>init.sh</code>

表 3: Agent 失败模式与双阶段对策的对应关系

系统性思维：问题导向的设计

这张对照表体现了一种优秀的工程实践——每个设计决策都直接对应一个具体的、已观察到的问题。这种“问题导向”的设计方法比“理论驱动”的方法更加实用，因为它确保每个机制都有明确的存在理由。在设计自己的 Agent harness 时，建议先系统性地收集和分类失败模式，再针对性地设计解决方案。

7.1 本章小结

通过系统性地识别 Agent 的失败模式并分别在 Initializer Agent 和 Coding Agent 层面设计对策，Anthropic 团队构建了一个完整的问题-解决方案矩阵。每个设计决策都有明确的问题背景，这

使得整个系统的设计理性且可解释。

8 设计原则提炼

从 Anthropic 的实践中，我们可以提炼出以下适用于所有长时间运行 Agent 系统的设计原则：

8.1 原则一：状态外部化

Agent 的工作状态不应仅存在于 context window 中，而应该被外部化到持久存储（文件系统、数据库等）中。这样即使 context window 被清空，新的 session 也能从外部状态中恢复。

8.2 原则二：任务分解与增量推进

复杂任务必须被分解为可以在单个 session 内完成的小任务。每个 session 的目标应该是”完成一个功能并留下干净的状态”，而非”尽可能多地完成任务”。

8.3 原则三：不可变的评判标准

任务完成的评判标准（如 feature list）应该在一开始就确定，并且在执行过程中不可被 Agent 修改。这防止了 Agent 通过降低标准来”完成”任务。

8.4 原则四：先验证后推进

每个 session 开始时应该先验证现有功能的正确性，再开始新功能的开发。这防止了在已有问题的基础上叠加更多问题。

8.5 原则五：工具即能力边界

Agent 的能力上限由其可用工具决定。要提升 Agent 的表现，首先应该考虑是否可以提供更好的工具，而不仅仅是优化 prompt。

与人类工程实践的对应

这些原则与人类软件工程的最佳实践高度一致：状态外部化对应文档和 wiki，任务分解对应 sprint planning，不可变标准对应 acceptance criteria，先验证后推进对应 CI/CD，工具即能力对应开发环境。这并非巧合——Anthropic 明确表示他们的灵感来自”优秀的软件工程师每天都在做的事情”。

8.6 本章小结

从 Anthropic 的实践中可以提炼出五条通用设计原则：状态外部化、任务分解与增量推进、不可变的评判标准、先验证后推进、工具即能力边界。这些原则不仅适用于编码 Agent，也可以推广到其他类型的长时间运行 Agent 系统。

9 未来方向与开放问题

9.1 单 Agent vs. 多 Agent 架构

Anthropic 指出, 目前尚不清楚单一的通用编码 Agent 是否在所有 context 下都能表现最佳, 还是通过多 Agent 架构可以实现更好的性能。可能的专业化 Agent 包括:

- **Testing Agent**: 专门负责编写和运行测试
- **QA Agent**: 专门负责质量保证和 bug 检测
- **Code Cleanup Agent**: 专门负责代码重构和清理
- **Architecture Agent**: 专门负责系统设计和架构决策

9.2 领域泛化

当前的方案主要针对全栈 Web 应用开发进行了优化。Anthropic 认为这些经验可能适用于其他领域, 包括但不限于:

- 科学研究中的实验设计和数据分析
- 金融建模与量化分析
- 系统运维和基础设施管理
- 文档撰写和内容创作

从 Web 开发到通用场景

Web 应用开发之所以成为首选实验场景, 是因为它具备几个有利条件: 功能可以被清晰地分解为独立单元, 端到端测试可以通过浏览器自动化实现, 且开发流程 (编码-测试-提交) 高度标准化。将这些实践泛化到其他领域时, 关键挑战在于如何定义”功能”和”测试”——在科学研究中, 一个”功能”可能是”验证一个假设”, 而”测试”可能是”重现实验结果”。

9.3 本章小结

长时间运行 Agent 的研究仍处于早期阶段。多 Agent 架构是否优于单 Agent, 以及这些实践如何泛化到 Web 开发之外的领域, 都是值得深入探索的方向。

10 总结与延伸

10.1 核心贡献

Anthropic 这篇工程博客的核心贡献在于提供了一套经过实践验证的、解决 Agent 跨 context window 工作问题的完整方案。其关键要素可以概括为:

1. **双阶段架构**: Initializer Agent 搭建环境, Coding Agent 增量推进
2. **Feature List**: 结构化的 JSON 格式功能清单, 防止 Agent 过早宣布完工
3. **Progress File + Git History**: 跨 session 的状态传递机制

4. **标准化启动流程**：减少每个 session 的”热身”成本
5. **端到端测试**：通过浏览器自动化确保功能真正可用

10.2 对 Agent 开发者的启示

对于正在构建长时间运行 Agent 系统的开发者而言，本文最重要的启示是：

不要只优化模型，更要优化 Harness

Agent 的表现不仅取决于底层模型的能力，更取决于围绕模型构建的 harness（运行框架）的设计。即使是最强的前沿模型，如果 harness 设计不当，也无法在长时间运行场景下保持有效输出。反之，一个精心设计的 harness 可以让模型的能力得到充分发挥。

10.3 拓展阅读

- Anthropic Claude Agent SDK 文档
- Anthropic Claude 4 Prompting Guide——关于多 context window 工作流的最佳实践
- 原文附带的 quickstart 代码示例
- Anthropic 关于 Agent 安全性的研究（长时间自主运行的 Agent 需要额外的安全考量）